CLONESCOPE

Source Preprocessing
Learning & Implementation Manual

Project: An Explainable Code Clone Tagging and Stylometric Analysis Tool

Student: Raian Huq (BSSE1708) Institute: IIT, University of Dhaka

Supervisor: Dr. Kazi Muheymin-Us-Sakib

Language: C++17 Scope: basic lexical source cleaning; no ML training

IMPLEMENTED	EVIDENCE	NEXT
File + folder preprocessing	5/5 focused smoke tests passed; CLI sample checks	Tokenization and function/block representation
Finite-state comment removal	`//`, `/*...*/`, ordinary literals	Further C++ lexical edge cases

Snapshot recorded: feature branch `feat/source-preprocessing`, published through commit `f523c1a`. The repository was checked on 27 October 2026 (Dhaka date).

Scope statement: This manual describes the working basic preprocessing implementation. It is not a full C++ lexer or the compiler's complete preprocessing phase; it does not implement clone tagging yet.

DEVELOPMENT PRINCIPLE

Learn the concept → implement the smallest useful change → compile → test behavior → commit meaningful progress → document limitations.

1. Purpose and Learning Outcomes

CloneScope must eventually compare program fragments from different submissions. Raw source text contains comments and formatting differences that may not be useful for clone matching. The current preprocessing module removes recognized comments while retaining ordinary source text for subsequent tokenization.

After studying this manual, explain how to

- Describe the difference between source preprocessing and full compilation.
- Explain a C++ header/implementation split and why multiple .cpp files must be linked.
- Trace the five-state scanner character by character.
- Explain lookahead, escape sequences, preservation of newlines, and index bounds.
- Demonstrate single-file and folder-input integration with C++17 streams and filesystem.
- Defend focused unit tests, known limitations, and meaningful Git commits.

Real workflow - as implemented

Stage	Responsibility
User input	A path to one source file or a local directory
Discovery	For a directory, visit immediate entries and filter regular lowercase .c/.cpp files
Read	Open each file via ifstream and collect its complete contents in a string
Preprocess	Call removeComments(...) on the source string
Report	Print cleaned source code; tokenization and clone results are not yet produced

Why the whole file matters: A block comment can start on one line and end on another. Passing each line to a new scanner would reset its state; passing the complete source string allows the state to survive line boundaries.

2. Project Structure and Core Interfaces

```
SPL_01_BSSE1708/
|-- src/
|   |-- main.cpp               CLI and filesystem input
|   |-- preprocessor.hpp       Public function declaration
|   `-- preprocessor.cpp       Lexical-state implementation
|-- samples/
|   `-- preprocess_example.cpp
|-- tests/
|   `-- preprocessor_smoke.cpp
|-- docs/
|   `-- learning-notes/
`-- README.md
```

Header vs implementation vs caller

File	Responsibility	Key syntax
preprocessor.hpp	Advertises the callable interface	std::string removeComments(const std::string& source);
preprocessor.cpp	Defines the algorithm	std::string removeComments(...) { ... }
main.cpp	Calls the function	std::string cleanedCode = removeComments(sourceCode);

```
#ifndef CLONESCOPE_PREPROCESSOR_HPP
#define CLONESCOPE_PREPROCESSOR_HPP
#include <string>
std::string removeComments(const std::string& source);
#endif
```

Line-by-line: The include guard prevents duplicate processing within a translation unit. `#include <string>` provides `std::string`. The return type is a string. `const std::string&` avoids copying the input when calling the function and prevents modification through that reference. The semicolon indicates a declaration, not a function body.

Why not put everything in main.cpp? The interface/implementation boundary separates file input from text transformation, making the preprocessor independently testable.

3. Lexical State Machine

An ordinary slash does not always start a comment. The scanner interprets characters in the context of its current lexical state.

State	Meaning	Typical transition
Normal	Outside comments and ordinary literals	// → LineComment; /* → BlockComment; quotation mark → literal state
LineComment	After // in Normal	Newline → Normal
BlockComment	After /* in Normal	*/ → Normal
StringLiteral	Inside double-quoted string	Unescaped double quote → Normal
CharLiteral	Inside single-quoted character literal	Unescaped single quote → Normal

```
enum class ScanState {
    Normal,
    LineComment,
    BlockComment,
    StringLiteral,
    CharLiteral
};

ScanState state = ScanState::Normal;
```

Why enum class? It defines one strongly typed variable containing exactly one named state at a time. Five unrelated Boolean flags could produce contradictory states.

Mental trace: two very different meanings of //

```
int x = 10; // ordinary comment
std::string url = "https://example.com";
```

For the first line, the scanner is in Normal at the two slashes and enters LineComment. For the second line, it enters StringLiteral at the opening double quote, so the slashes in https:// are ordinary string contents. A quote closes a literal only if the scanner does not treat it as escaped.

4. Inside removeComments(): New C++ Concepts

A. Index-based traversal and safe lookahead

```
for (std::size_t i = 0; i < source.size(); ++i) {
    char current = source[i];
    if (current == '/' && i + 1 < source.size() &&
        source[i + 1] == '/') {
        // Enter LineComment, replace the two marker characters
    }
}
```

`std::size_t` is the standard unsigned size/index type. `source[i]` returns the character at index i. The loop condition protects the current index. `i + 1 < source.size()` protects the next-character lookup. The && operator short-circuits, so the lookahead expression is not evaluated when the bound check fails.

B. Preserve output while scanning

```
std::string result;
result += current;  // append an ordinary source character
result += "  ";      // replace a two-character delimiter
++i;                // skip a character already handled by lookahead
return result;      // after the full loop, not inside it
```

The function creates a new output string rather than changing the input. For recognized comment text, it generally appends spaces instead of deleting bytes. This helps keep source positions stable for later evidence; newline characters are preserved in the tested LF cases.

C. The difference between assignment, comparison and early exit

Syntax	Meaning
state = ScanState::Normal;	Assign a new value to the state variable
state == ScanState::Normal	Check whether the state equals Normal
return result;	Exit the function, sending back the result
continue;	Skip to the next iteration of a loop (used in main.cpp on file-open failure)

5. How the Five States Behave

Normal: decide whether a character begins a special region

```
if (current == '/' && i + 1 < source.size() &&
    source[i + 1] == '/') {
    state = ScanState::LineComment;
    result += "  ";
    ++i;
} else if (current == '/' && i + 1 < source.size() &&
           source[i + 1] == '*') {
    state = ScanState::BlockComment;
    result += "  ";
    ++i;
}
```

The two delimiter characters are replaced with two spaces and processed together. Ordinary characters are appended unchanged. Opening double/single quotes enter the corresponding literal states.

LineComment: continue until newline

```
else if (state == ScanState::LineComment) {
    if (current == '\n') {
        result += '\n';
        state = ScanState::Normal;
    } else {
        result += ' ';
    }
}
```

The newline is preserved and returns the scanner to Normal. Other recognized line-comment characters become spaces.

BlockComment: continue until */

```
else if (state == ScanState::BlockComment) {
    if (current == '*' && i + 1 < source.size() &&
        source[i + 1] == '/') {
        result += "  ";
        state = ScanState::Normal;
        ++i;
    } else if (current == '\n') {
        result += '\n';
    } else {
        result += ' ';
    }
}
```

The explicit end delimiter, rather than a newline, terminates a block comment. This lets a single block comment span many lines.

StringLiteral and CharLiteral: preserve escaped delimiters

```
else if (state == ScanState::StringLiteral) {
    if (current == '\\' && i + 1 < source.size()) {
        result += current;
        result += source[i + 1];
        ++i;
    } else if (current == '"') {
        result += current;
        state = ScanState::Normal;
    } else {
        result += current;
    }
}
```

The character-literal handler follows the same pattern but checks the single-quote delimiter. The scanner appends backslash plus the following character as a pair and skips the latter, so an escaped quote is not interpreted as a closing quote.

Important: The above snippets describe the implemented simple lexical behavior, not complete C++ translation semantics.

6. Integration in main.cpp

Single-file input

```
#include <sstream>
#include "preprocessor.hpp"

// after successfully opening inputFile:
std::ostringstream buffer;
buffer << inputFile.rdbuf();
std::string sourceCode = buffer.str();
std::string cleanedCode = removeComments(sourceCode);
cout << "Preprocessed source code:\n";
cout << cleanedCode;
```

Read line by line vs read completely: The previous getline loop printed each line immediately. The new stream buffer copies the remaining file contents into an output string stream, then buffer.str() retrieves one string. That string is passed once to removeComments().

Folder input

```
for (const auto& entry : fs::directory_iterator(sourcePath)) {
    if (entry.is_regular_file() &&
        (entry.path().extension() == ".c" ||
         entry.path().extension() == ".cpp")) {
        std::ifstream sourceFile(entry.path());
        if (!sourceFile.is_open()) {
            std::cerr << "Error: Could not open file\n";
            continue;
        }
        std::ostringstream buffer;
        buffer << sourceFile.rdbuf();
        std::string cleanedCode = removeComments(buffer.str());
        std::cout << cleanedCode << '\n';
    }
}
```

This is a concise explanatory sketch, not a verbatim replacement for main.cpp. A new stream and a new call to removeComments() are created for every matched file. Each call resets its own ScanState to Normal. The implemented directory traversal is non-recursive and filters lowercase .c/.cpp only.

Output behavior and limitations

- The CLI currently prints preprocessed code; it does not yet extract functions, tokenize code, assign clone tags, or calculate pairwise similarity.
- If the file cannot be opened, the single-file path returns a nonzero exit code; in a folder, processing continues to other files.
- Full-file buffering costs memory proportional to the file size. Very large inputs will need resource limits and more systematic I/O error checking.

7. Testing: What Passed, and What That Proves

A smoke test is a small, repeatable check of an essential behavior. The current suite uses exact string comparisons and stops on the first failure (fail-fast).

Test	Case	Observed result
1	// comment removed while source and newline remain	PASS
2	/*x*/ replaced with five spaces inside a code line	PASS
3	URL and /*...*/ inside a normal string remain unchanged	PASS
4	Escaped double quotes do not end a normal string	PASS
5	Character literals, including an escaped single quote, are preserved	PASS

```
g++ -std=c++17 -Wall -Wextra \
  tests/preprocessor_smoke.cpp src/preprocessor.cpp \
  -o preprocessor_smoke.exe

.\preprocessor_smoke.exe
```

Interpretation: The five cases passed when run in VS Code. This demonstrates behavior for those inputs only; it is not a mathematical proof of a fully correct C++ comment-removal algorithm.

Manual end-to-end checks

- Single-file input: samples/preprocess_example.cpp - actual comments were visibly replaced; URL, faux comment inside a string, escaped quotes and a character literal were visibly preserved.
- Folder input: samples - hello.c and preprocess_example.cpp were discovered and processed; output showed supported-source filtering and separate preprocessing.
- The snapshots supplied show the relevant output portions; additional automated folder-regression coverage is a future improvement.

Good next tests before trusting larger inputs

- Block comments spanning multiple lines with CRLF newlines.
- An empty source file; a comment ending at end of file; an unterminated /* comment.
- Raw string literals that contain //, /*, or quotes.

- Escaped line endings, backslash-newline splicing and preprocessor directives.

8. Build, Linking and Debugging

Command	Purpose
g++ -std=c++17 -Wall -Wextra -fsyntax-only src/preprocessor.cpp	Compile check of the preprocessing implementation without producing a program
g++ -std=c++17 -Wall -Wextra src/main.cpp src/preprocessor.cpp -o clonescope.exe	Build and link the complete current CLI
.\clonescope.exe	Run the CLI from the current PowerShell directory
git status -sb	See current branch and working-tree status

The linker error we learned from

Building main.cpp alone after adding removeComments() caused an `undefined reference` error. The header made the function declaration known to the compiler, but the link step did not include the function's definition in preprocessor.cpp. Adding both .cpp files to the build command resolved the error.

Viva wording: Compilation checks/produces object code from source; linking combines objects and resolves references across implementation files.

9. Git Evidence and Meaningful Checkpoints

Feature branch: `feat/source-preprocessing`. The remote branch was verified published and four commits ahead of main after the folder-input integration.

Short commit ID	Purpose
0f925a8	Scaffold the header, implementation skeleton and sample test source
78787c2	Implement basic lexical-state comment handling with tests
d616c40	Integrate preprocessing into single-file input
f523c1a	Integrate preprocessing into folder input

Why four commits? Each reflects a coherent, understandable development step; they are not arbitrary activity markers. Using a branch keeps stable main separate until the component is reviewed and merged.

Git concepts explained

- Stage: Select which modifications belong to the next snapshot.
- Commit: Record a version in local repository history.
- Push / Sync: Publish commits to the corresponding GitHub branch.
- Pull request: Propose reviewable integration from feature branch into main.

- Merge: Incorporate the feature branch into the main branch after checks.

A PDF documentation commit and README update can be made after this manual is copied into docs/learning-notes. Do not claim those actions have happened until they are actually committed and verified.

10. Known Limitations: Explain These Clearly

Area	Current boundary
Raw strings	R"(...)" constructs are not parsed as raw strings; comment-like sequences in them can be misinterpreted.
Backslash-newline	The full C++ translation phase for line splicing is not modeled; a // comment can continue across a physical newline.
CRLF handling	The scanner explicitly preserves LF (\\n). It may replace CR (\\r) with a space inside comments, so raw byte/column preservation is not general.
Lexical diagnostics	Malformed/unclosed literals and unterminated block comments are not reported as syntax errors.
Preprocessor directives	This algorithm does not expand macros, process includes or implement conditional preprocessing.
Input discovery	Only immediate directory entries, and lowercase .c/.cpp file extensions, are included.
Clone analysis	No tokenization, function/block extraction, Type-1/Type-2 matching or stylometry yet.

Correct framing for the supervisor: This is a verified basic comment-cleaning stage in a larger explainable clone-analysis pipeline, with deliberately documented limitations. It should not be described as a complete, standards-compliant C++ lexer.

Immediate next development milestone

Start tokenization on preprocessed source, define stable token representations and source-location mapping, then move toward function/block segmentation and explainable Type-1/Type-2 candidate tagging. Do not infer plagiarism automatically from similarity.

11. Supervisor Viva: Practice Answers

Q. What does preprocessing do in CloneScope?

A. It prepares source text for later comparison by removing supported comments while preserving code content and usable line structure.

Q. Why do comments matter for clone tagging?

A. Formatting and comments can differ between otherwise similar code. Handling irrelevant differences can improve subsequent comparison.

Q. Is source cleaning the same as the C++ preprocessor?

A. No. We do not evaluate #define or #include. We currently perform a limited lexical transformation for analysis.

Q. What is a finite-state machine?

A. A model with a finite set of states whose transitions depend on input. My scanner has Normal, two comment states and two literal states.

Q. Why enum class?

A. It keeps the scanner in one strongly typed named state and avoids impossible combinations of several Boolean flags.

Q. Why look ahead to source[i+1]?

A. Comment openers and closers consist of two characters: //, /* and */. Lookahead identifies them safely.

Q. How do you prevent out-of-bounds access?

A. I check that i+1 is less than source.size() before reading source[i+1]; && short-circuits the access.

Q. Why increment i inside the loop?

A. When two characters are processed together, ++i skips the second one so it is not processed twice.

Q. Why preserve newline characters?

A. It helps retain original source line relationships for later evidence and reporting.

Q. How do you avoid removing https://?

A. The scanner enters StringLiteral after an opening double quote and keeps // inside the string unchanged.

Q. Why handle escaped quotes?

A. An escaped quote is not a closing delimiter, so we preserve it with the preceding backslash.

Q. Why call removeComments once per file?

A. The function begins in Normal for each call. Processing each file separately prevents state leakage between files.

Q. Why use ostringstream?

A. It collects complete file contents into one std::string, allowing state to survive across line breaks.

Q. How do you know the feature works?

A. Five focused smoke tests passed, and single-file and folder-input runs visibly produced the expected cleaned output.

Q. Does five passes prove complete correctness?

A. No. It validates five specific inputs; raw strings, line splicing and other edge cases still need tests and possibly code changes.

Q. Why were there several small commits?

A. Each records a coherent tested milestone: scaffold, algorithm, single-file integration and folder integration.

Q. What is the next part of CloneScope?

A. Tokenization, followed by function/block representation and Type-1/Type-2 clone candidate detection with explainable evidence.

Q. Can CloneScope declare plagiarism?

A. No. Similarity evidence is for human review. It must not independently label a student guilty of plagiarism.

Study tip: To prepare for the supervisor, explain each answer once in your own words, then open the matching code and point to the exact statement implementing the concept.

Repository reference: https://github.com/raian-x8/SPL_01_BSSE1708